Group 2311 – Restricted Boltzmann Machines

Marina Fedon Vocaturo, Rodrigo Golan, Tommaso Guercio, and Niccolò La Delfa
(Dated: April 21, 2026)

The report focuses on the Restricted Boltzmann Machine (RBM), an unsupervised generative model able to draw conclusions about unknown data. The aim is to analyse the correlations between the visible and hidden units, study how the algorithm learns the probability distribution and improve the learning process through the use of different data encoding and training techniques, such as centering trick, gradient manipulation, and number of contrastive divergence steps. The results are validated using the difference in the mean energy of the generated and original samples and the feedback given by the log-likelihood function. Findings reveal that spin notation, combined with Adam gradient descent algorithm, centering the gradient and just one CD step lead to the best performances of the RBM.

INTRODUCTION

The Boltzmann Machine is a generative unsupervised model that relies on the learning of a probability distribution from a unique dataset and the use of that distribution to draw conclusions about unexplored data. The Boltzmann Machine has one or more hidden layers, with $\{h_\mu\}$ as units, in addition to the input layer, also known as the visible layer, with units $\{v_i\}$ (see Figure 1).

In the report we focus on a particular case, the so-called Restricted Boltzmann Machines (RBMs). The term 'restricted' means that we are not permitted to connect two types of layers that are of the same type to one another, therefore the only connections allowed are between the visible and hidden layers.

The aim of the project is therefore to analyse the correlations between the visible and hidden units in the RBMs, see how the algorithm learns the probability distribution through the study of the weights and bias, and how we can improve the learning process through the use of different techniques.

In our specific case, the data has an unknown structure composed by a set of binary vectors v with $G = 5$ one-hot blocks. Each block contains $A = 6$ elements: one of them is set to 1 and all the others are set to 0. Another possible choice explored in this work is the spins encoding where we change the values fixed to 0 with -1.

A large portion of the project focuses on improving the model training by comparing several gradient descent algorithms, implementing data manipulations tricks and changing other hyper-parameters. The last section of the project involves the analysis of the results given by the optimal configuration.

METHODS

To allow our model to analyse the data and extract information from it, we need to establish how to train the RBM. Firstly, to monitor the training of the RBM, we randomly split data into a training (80%) and a test (20%) set.

The RBM is trained with the Maximum Likelihood Estimation (MLE). This approach involves maximising the log-likelihood using different Stochastic Gradient Descent (SGD) methods: from the basic vanilla gradient descent to the more sophisticated Adam and RMSprop. The log-likelihood gradient takes the following form [1]:

$$\begin{aligned}
\frac{\partial \mathcal{L}(\{W_{i\mu}, a_i, b_\mu\})}{\partial W_{i\mu}} &= \langle v_i h_\mu \rangle_{data} - \langle v_i h_\mu \rangle_{model} \\
\frac{\partial \mathcal{L}(\{W_{i\mu}, a_i, b_\mu\})}{\partial a_i} &= \langle v_i \rangle_{data} - \langle v_i \rangle_{model} \\
\frac{\partial \mathcal{L}(\{W_{i\mu}, a_i, b_\mu\})}{\partial b_\mu} &= \langle h_\mu \rangle_{data} - \langle h_\mu \rangle_{model}
\end{aligned} \tag{1}$$

To compute the expectation values we exploit the fact that there are no interactions of visible units with themselves or hidden units with themselves. The visible and hidden units are therefore conditionally independent, with [1]:

$$p(v_i = 1|\mathbf{h}) = \sigma(a_i + \sum_\mu W_{i\mu} h_\mu) \tag{2}$$

$$p(h_\mu = 1|\mathbf{v}) = \sigma(b_\mu + \sum_i W_{i\mu} v_i) \tag{3}$$

where $\sigma(z) = 1/(1 + e^{-z})$. To obtain the hidden units and the model visible units samples, we use an approximate Gibbs sampling technique called Contrastive Divergence (CD). The idea is to iteratively sample from the conditional distributions so that $\mathbf{h}_{t+1} \sim p(\mathbf{h}|\mathbf{v}_t)$. For visible units we implement instead a backward step of contrastive divergence that preserves the one-hot encoding structure. The samples are therefore generated by extracting blocks of A bits with probability proportional to the Boltzmann weight of its one-hot encoding.

In CD we just perform n iterations of (block) Gibbs sampling, with n taken between 1 and 10 in our case. The price for this truncation is that we are not drawing samples from the true model distribution, but for our purpose it has proven to work reasonably well.

Additionally, to improve the performance of the algorithm, we implement the so-called "centering trick" [2], with two different strategies. The first one consists in shifting the visible and hidden variables by some offset parameters. The latter consists in centering the gradients.

Finally, we use two methods to verify the validity of the results. The first is to calculate the difference be-

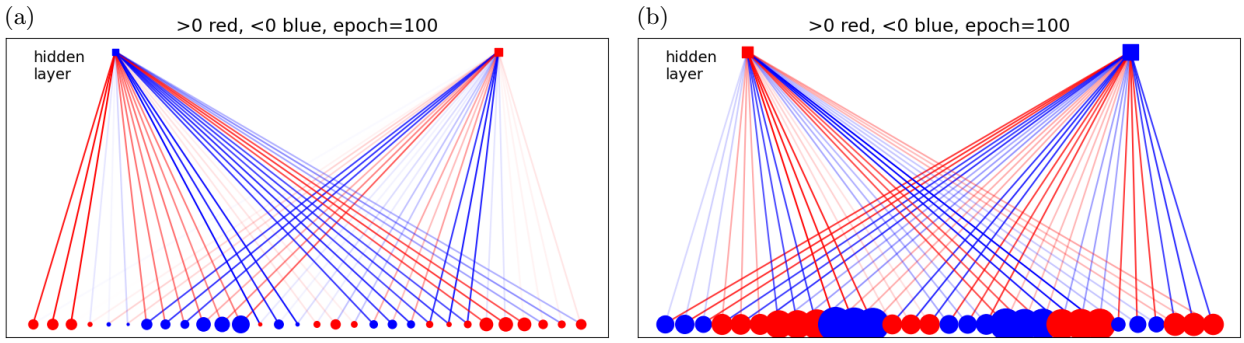

FIG. 1: Panel (a) shows the connections between the two hidden units on top and the 30 visible units below for the case of spin notation after a basic training. Positive and negative weights, presented with lines are respectively marked in red and blue while colour intensity encodes the magnitude. The circles represent the local biases, whose magnitude is encoded in the size of the dot. Panel (b) reports instead the case of binary notation.

tween the mean energy values of the data and the model: $E_{diff} = |\langle E\rangle_{data} - \langle E\rangle_{model}|$. The second involves the calculation of the log Likelihood:

$$\mathcal{L}(\{W_{i\mu}, a_i, b_\mu\}) = \langle E(\mathbf{h}, \mathbf{v}; \{W_{i\mu}, a_i, b_\mu\})\rangle_{data} - logZ(\{W_{i\mu}, a_i, b_\mu\}) \tag{4}$$

where:

$$E(\mathbf{h}, \mathbf{v}; \{W_{i\mu}, a_i, b_\mu\}) = -\sum_i a_i v_i - \sum_\mu b_\mu h_\mu - \sum_{i,\mu} v_i W_{i\mu} h_\mu a \tag{5}$$

$$Z(\{W_{i\mu}, a_i, b_\mu\}) = \sum_{\{\mathbf{v}\},\{\mathbf{h}\}} e^{E(\mathbf{h},\mathbf{v};\{W_{i\mu},a_i,b_\mu\})} \tag{6}$$

RESULTS

A first evaluation of the performance of the RBM regards the choice of input data encoding. The use of binary values 0,1 or spins -1,+1 might produce different results. In general, the use of binary units is good for learning correlation between features, whereas spins are more suitable to capture anti-correlations. When data are unknown, it is beneficial to try both options to avoid poor performance of the RBM. A visual comparison between the two different conventions is reported in Figure 1. The main advantage coming from the binary notation is the clearer visualisation and interpretation in the weights relating hidden and visible units. The sign directly defines an excitatory or inhibitory connection and the magnitude is related to the strength of the connection. On the contrary, for the spin notation, the magnitude does not necessarily correspond to the strength, since it has to compensate for the -1, + 1 constraint on the values. However, apart from visualisation, the use of the log likelihood estimator allows us to notice a little improvement in the spin case with respect to the binary one. Thus, we use the former notation for all training procedures.

The training and test sets show a compatible result across all training schemas, indicating a likely absence of overfitting. Therefore, we choose to omit a regularisation term in the training procedure. This choice conveys benefits from an interpretative standpoint in reading the weights, as close to 0 values now indicate clear sign of no influence between the two given units.

The comparison between different optimizers showcases Adam's superiority as an overall better choice. The results for the three alternatives are presented in Fig. 2. Concerning the energy difference estimator, the use of Adam shows a faster convergence at lower absolute values. Looking instead at the log likelihood as a function of the epoch, we can notice that the vanilla case is characterised by a strongly oscillating trend, while Adam and RMSprop share a similar behaviour with a regular increase in $\mathcal{L}$ up to a maximum and a slow decrease in the very last epochs.

Another key issue to take into account is the number of CD steps. The optimal choice is estimated by finding a balance between performance and training speed. Energy and $\mathcal{L}$ estimates for an increasing number of iterations are reported in Fig. 2. We can see from graph (d) that the energy difference curves are very similar to each other, and therefore the improvement given by increasing the CD steps seems to be practically irrelevant. On the other hand, graphs (e) and (f) show us that by increasing the number of CD steps the performance of the model deteriorates. In fact, we can see that the $\mathcal{L}$ curves with 2 or more steps have a decreasing trend from a certain point onwards. This could be because with more CD steps, the model spends more time in the fantasy distribution before updating the parameters, which can cause it to become more and more dissimilar to the true data distribution. Thus, the model can get stuck in a local optima causing a poorer fit of the training data.

The difference between the log-likelihoods for the data and fantasy distributions is also an important metric to consider. Closer $\mathcal{L}$ for the two distributions exhibit the model's ability to generate fantasy data similar to the training. In this case, the $\mathcal{L}$ estimates for the fantasy distribution are consistently lower than those for the original distribution, indicating that the model has not fully

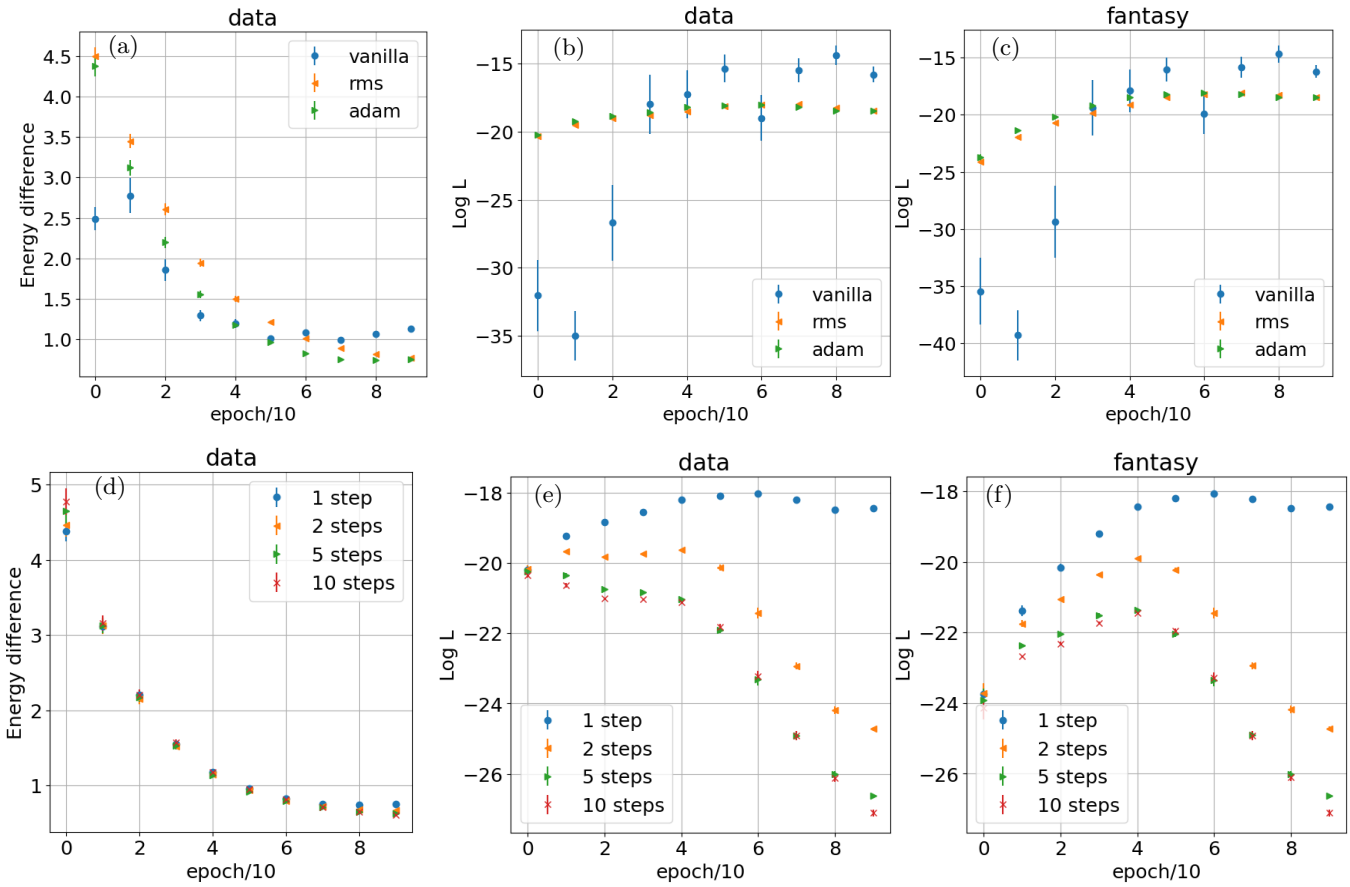

FIG. 2: Panel (a) compares energy differences as a function of the epoch for the use of different gradients. While panels (b) and (c) compare the $\mathcal{L}$ respectively for real and fantasy data for different gradients. Both energies and $\mathcal{L}$ are average values calculated every 10 epochs. We can notice a little decrease in $\mathcal{L}$ for fantasy data with respect to the other case. Panels (d), (e) and (f) show respectively energy difference estimates and $\mathcal{L}$ for real and fantasy data for the choice of Adam gradient and varying the number of CD steps.

grasped the structure underneath the training data. This could be due to overfitting, because the model captures noise and specific patterns rather than general features. To help prevent overfitting we can implement regularization in the model training, such as adding a penalty term proportional to the value of the weights to the cost function ($\mathcal{L}$) or by randomly dropping some units of the RBM (dropout).

Furthermore, the training procedure can also be improved by taking advantage of the centering trick, which can boost the convergence rate and stability of the algorithm. The whole trick relies on the calculation of local averages to compute the shifts; thus, it will have different effects depending on the batch size. If the mini batch is too small, the estimate of the mean might not be accurate, reducing the effectiveness of the trick. For very large batch size, the variance in the gradient increases leading to slower convergence of the algorithm. The optimal mini batch size is set to 500, having enough statistics to accurately estimate the mean while remaining computationally efficient. In general, the application of the trick on the parameters or on the gradient leads to different results, as shown for the best case (Adam,CD1) in Figs. 3 (a),(b) and (c). The difference in energy behaviour with and without centering is minimal; however, its impact is evident in the $\mathcal{L}$ estimates. The increase in $\mathcal{L}$ is enhanced when the trick is applied and the centering of the gradient overcomes the alternative on the parameters. A similar behaviour can be found also using other types of gradients, also in the rms case the centering of the gradient seems more advantageous, while for the vanilla one the centering of parameter is more effective.

After choosing the optimal conditions, the results of the training procedure can be analysed. Figure 3(d) shows a graphical representation of visible biases and weights for each category depending on the position of the group. The bias can be interpreted as the likelihood of the corresponding unit to be active in absence of any input from other units. In particular, visible units placed in groups 1,2,3 have almost all negative biases, implying that other visible units more correlated with input data are more likely to be activated, such as those placed in groups 0 and 4. This also means, possibly, that they act by filtering out noise that does not align with the learnt patterns in the input data. The weight matrices can be interpreted by assuming that the probability of picking a given category depends on w_{ij}. Consider, for instance, the first hidden unit and group 1, when $h_1 = +1$

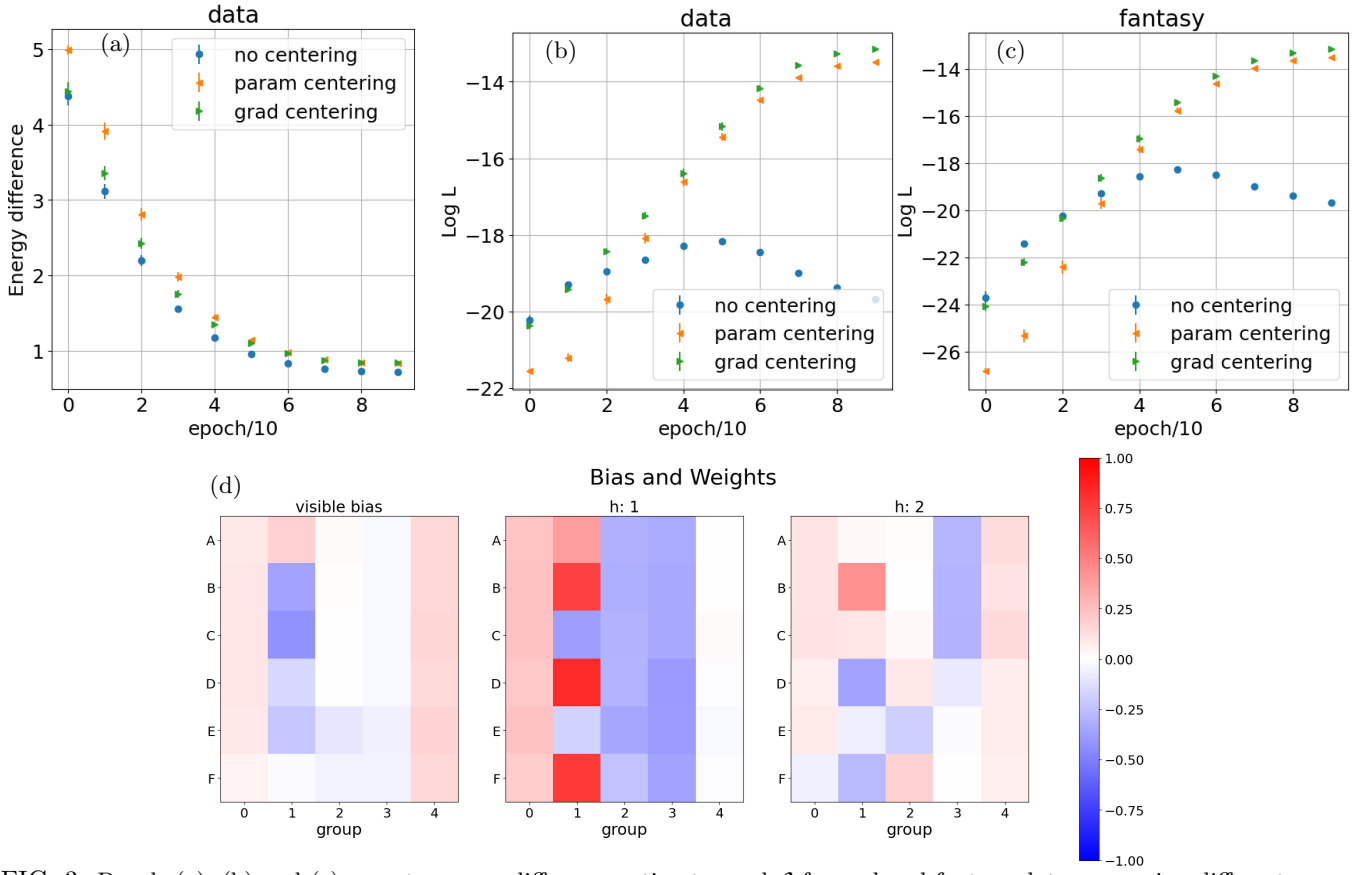

FIG. 3: Panels (a), (b) and (c) reports energy difference estimates and $\mathcal{L}$ for real and fantasy data comparing different centering options. While panel (d) shows visible biases and weights connections to each hidden unit h. Each matrix relates the 6 different categories (marked with letters), with all the possible group positions. Colours encode the magnitude of each weight and bias.

categories corresponding to w_{i1} significantly larger than zero (red blocks for B,D,F) are more likely to be chosen and there is inhibition for $w_{i1} < 0$. On the contrary, if $h_1 = -1$ the categories with $w_{i1} < 0$ (i.e. C,E) are chosen more frequently. Concerning weights, visible units in groups 2,3 have almost all negative or nearly zero values in both matrices, implying anticorrelation with both hidden units. This may have an impact on the reduction of noise and spurious correlations in the input data. Looking at weights connected to the first hidden unit, one can notice that visible units of categories B,D,F have strong positive values when placed in group 1, and the opposite happens for D and F in the third group. This suggests an anticorrelation between these categories for groups 1 and 3. Considering the matrix for the second hidden unit one can notice that absolute values of weights are in general lower, suggesting that the second hidden unit is less likely to activate in general, regardless of which visible units are active. For $W_{i,\mu} \sim 0$, since we chose not to perform any regularisation, this implies genuinely that there is no direct influence of v_i on the activation of h_μ. In general, one can notice that weights connecting visible units of the same group to the different hidden units are significantly dissimilar, this is evident especially for group 1 (where some of the most significant categories change sign going from h_1 to h_2) but also groups 2 and 4. This means that the two hidden units probably learnt successfully to recognise two different patterns in the data.

CONCLUSIONS

We show the usage of RBM to learn the probability distribution of a unique dataset and use that information to draw conclusions about unexplored data. Specifically, the plot of the weights showcases different learning patterns for each of the hidden units, corresponding to different features of the original data. Further investigation on the most effective number of hidden units for this particular dataset should be performed, although increasing N_h while maintaining the size of the relatively small data size should lead to poorer results. Another avenue for investigation would be to examine the trade-off between regularization and weights interpretation. Our results indicate that the centering trick applied to the gradient appreciably improves the learning process and that the spin encoding gives better results.

[1] P. Mehta, M. Bukov, C. Wang, A. G.R. Day, C. Richardson, C. K. Fisher, D. J. Schwab A high-bias, low-variance introduction to Machine Learning for physicists, Physics Reports **810**, 87–101 (2019)

[2] M. Bortoletto Study of performances for Restricted Boltzmann Machines [Master thesis, Università degli studi di Padova]